

VAAK Execution Sheet

Companion to the VAAK brief, second issue. Subalakshmi and Shiva. Follow this in order.

Issued 13 September 2026

What this document is. The brief says what the work claims and why. This sheet says exactly what to do, in what order, and what file each step must leave behind. Work through it from the top. Every step names the artifact it produces, the condition that closes it, and the situation in which you stop and report instead of continuing. If a step does not close, do not move to the next one, and do not invent a way around it. Reporting a blocked step is correct behaviour and costs the project nothing. Working around a blocked step quietly is what costs the project everything, because every number produced after it is unusable and nobody knows.

How to work

1. **Nothing is optimised.** Every component is used as published, with default settings. If a result looks poor, that is the finding. Improving a component to make the number better destroys the claim, which is about what a developer obtains from available tooling today.
2. **No substitutions.** If a model, endpoint or repository named in the brief does not work, report it. Do not replace it with a similar one you found.
3. **The instance set is frozen once written.** After it is fixed, it is never edited, filtered or reordered, not even to drop an instance that keeps failing. A failing instance is data.
4. **Everything you actually run is recorded.** Every command, model name, version string, endpoint and random seed goes into config.md, copied exactly as run, at the moment you run it. Not reconstructed afterwards.
5. **One change at a time.** If you change two things and the number moves, you have learned nothing.
6. **Bad numbers get reported the same as good ones.** A flat result is a result and has a place in the paper. A hidden one does not.
7. **If you find yourself making a decision, stop.** Choices about languages, arms, scope and framing are listed in the brief under Decisions to make. Those come back as proposals, not as things you settle at the keyboard.
8. **Two failed attempts is the limit.** If a step will not close after two honest attempts, write down what you tried and what happened, and report it.

Directory layout

Create this once, at the top of the project, and put every artifact in its named place. Do not keep files in your home directory or in a scratch folder.

vaak/

config.md	every command, model, version, seed
notes.md	what went wrong, what you tried
data/instances.json	the frozen instance list
spans/	hand labelled spans, detector output
spokenized/	spoken form of each issue
translated/	masked, translated, restored
audio/	synthesized clips
transcripts/	recognition output
runs/	raw agent output, one folder per condition per run
results/	the csv files this sheet asks for

Part A. Set up and prove the setup works

1. Read the brief, then this sheet, end to end, before touching anything

You are looking for two things: what each condition is for, and which questions are not yours to settle. If a sentence in the brief does not make sense, write the sentence into notes.md and ask. Do not guess and proceed.

Produce	notes.md, opened, with any sentence of the brief you could not follow
Done when	You can state in your own words what C1, C1b, C2, C3 and C4 each control for
Stop and report if	You cannot say what C2 is for. That one is the control the whole attribution rests on

2. Install the host benchmark and the text agent

Clone SWE-bench/SWE-bench and SWE-agent/SWE-agent and install each following its own README. Use SWE-bench Verified, not full SWE-bench.

Record in config.md the commit hash of each repository, the Python version, and the model and version you configure SWE-agent with. SWE-agent is configured by a single YAML file. Put a copy of that YAML in config.md too.

Produce	config.md with both commit hashes, environment details, and the agent YAML
Done when	The evaluation harness runs to completion on the repository's own example and produces a score
Stop and report if	The harness does not run on the example. Do not continue into our own data with a broken harness

3. Freeze the instance set

Take SWE-bench Verified. Sort instance ids as plain strings, ascending. Take the first 50. Write them to data/instances.json as a JSON list of strings and do not touch the file again.

Sorting rather than sampling is deliberate here. It is reproducible without a seed and it is defensible in the paper, and the sampling frame for the scaled run is a separate research task that has not been done yet.

Produce	data/instances.json, a list of 50 instance ids
Done when	The file exists and contains exactly 50 ids
Stop and report if	Nothing here should block. If it does, something is wrong with the dataset download

4. Run C1, the English text baseline

Run SWE-agent on the 50 instances with the original issue text, unchanged. Run it three times over the whole set. Keep raw output under runs/c1_run1, runs/c1_run2, runs/c1_run3.

Score each run with the SWE-bench harness. Write results/c1.csv with one row per instance per run and columns instance_id, run, resolved, where resolved is 0 or 1.

Produce	results/c1.csv, and three folders under runs/
Done when	All three runs complete and the mean resolve rate across the three is computed and written into notes.md
Stop and report if	The resolve rate is zero, or all three runs are identical to the instance. Both suggest the harness is not scoring what you think it is

Part B. The three MUST items, before anything is built

Nothing in Part C can start until all three of these close. They are the definitions that everything downstream is instrumented against.

5. MUST. Read reference [3] in full, appendices included

This is the From Text to Voice preprint, arXiv:2605.15104. Answer these questions in writing, each with the table or section number you found it in.

One. Does the paper evaluate a cascade of recognition followed by a text model, or only omni models taking audio directly? The brief's first issue assumed a cascade arm exists. The abstract describes seven omni modal models, so this may be wrong, and if it is wrong our C3 has no precedent and needs its own justification.

Two. The text to voice gap is quoted as 1.8 points for one model and 4.8 for another, on one of the two benchmarks. Which benchmark, which models, and what is the gap on the other benchmark?

Three. What exactly do they report about argument value errors, with the number and the table?

Four. What is the clean audio content faithfulness figure, and how was it measured?

Five. What do they report about environmental noise, at what signal to noise levels?

Produce	notes.md section headed 'Reading of [3]', five answers, each with a table or section number
Done when	All five answered from the paper itself, not from the brief's summary of it
Stop and report if	Question one comes back as 'no cascade arm'. That changes the design and Ch decides what follows

6. MUST. Define the protected span taxonomy

Write a definition for each category: identifier, file path, API name, exception type, command flag, version string. Each definition says what is included, what is excluded, and gives two examples of each drawn from the actual 50 issues.

Then hand label 20 of the 50 issues against those definitions. Write spans/gold_spans.jsonl, one JSON object per line, with keys instance_id, span_text, span_type, char_start, char_end.

Label independently first, both of you, on the same 20, then compare and resolve every disagreement by amending the definition, not by picking a winner. A disagreement is a hole in the definition.

Produce	spans/taxonomy.md and spans/gold_spans.jsonl covering 20 instances
Done when	Both of you label the same 20 and every disagreement has been resolved by an amendment to the written definition
Stop and report if	A category keeps producing disagreements after two amendments. That is a real definitional problem and Ch should see it

7. MUST. Define identifier preservation rate formally

Write the formula. It is the proportion of protected spans that survive a stage byte identical. Then answer, in writing, the cases the formula does not obviously cover.

Does a case difference count as survival? Does a partial match count, and if so how is partial measured? What happens when the same span appears more than once in one instruction and survives in one place only? What happens when a span is dropped entirely rather than corrupted?

Then the hard one. In the omni arm there is no transcript, so there is nothing to compare against. Define the indirect procedure you will use to say whether a span survived. Write it down before you build anything, because a metric defined after seeing results is not a metric.

Produce	spans/metric.md, with the formula, the edge cases, and the omni procedure
Done when	A person who did not write it can compute the metric from the document alone
Stop and report if	You cannot find a defensible indirect procedure for the omni arm. Report it rather than inventing one

Part C. The probe, in order

Each step here feeds the next. Do not run ahead. Every artifact uses the file naming given, because the scoring scripts will assume it.

8. Build the span detector and measure what it misses

Write the component that finds protected spans automatically, following spans/taxonomy.md. Run it over the 20 hand labelled instances and compare against spans/gold_spans.jsonl.

Write results/masking_misses.csv with columns span_type, gold_count, detected_count, missed_count, false_positive_count. Then categorise the misses: read every missed span and give it a reason. Write the reasons and their counts into notes.md.

The number that matters is the miss count by category, not the overall rate. A detector that misses every exception type is a different problem from one that misses a few of everything.

Produce	the detector, results/masking_misses.csv, and the categorised miss list in notes.md
Done when	Misses are counted and categorised from reading them, not estimated
Stop and report if	The detector misses more than a handful in any one category and you cannot see why

9. Build spokenization and produce the spoken English form

For each of the 50 instances, rewrite the written issue into what a developer would plausibly say aloud to an agent. Shorter, spoken register, no markdown, no code blocks laid out on the page, but every protected span still present and still exact.

Write spokenized/{instance_id}.json with keys instance_id, original_issue, spoken_en, and notes for anything you were unsure about.

Check by reading ten of them aloud. If a sentence is not something a person would say, it has not been spokenized, it has been paraphrased.

Produce	spokenized/ with 50 files
Done when	All 50 exist and the span detector finds the same protected spans in spoken_en as in original_issue
Stop and report if	Spans are disappearing during spokenization. Fix that before going further, it contaminates everything

10. Run C1b, the contamination control

Run SWE-agent on the spokenized English text, three runs, same instances. Score as before. Write results/c1b.csv with the same columns as c1.csv.

Compare the mean against C1. This is the check that spokenization has not destroyed task information.

Produce	results/c1b.csv and the C1 against C1b comparison in notes.md
Done when	The comparison is computed and written down, with the difference stated
Stop and report if	C1b is materially below C1. Every downstream number is then contaminated and the whole probe pauses here

11. Build identifier protected translation

Replace every detected span with a placeholder of the form <S0>, <S1> and so on, keeping a map from placeholder to original text. Translate the carrier sentence with IndicTrans2. Restore the placeholders with the original text, byte identical.

Write translated/{instance_id}.json with keys instance_id, lang, masked_en, translated_masked, translated_restored, and spans as a list of objects with keys placeholder and text.

Check that every placeholder that went in comes back out. A placeholder that the translator mangled or dropped is a bug in this stage, not a finding about speech.

Produce	translated/ with 50 files for the probe language
Done when	Placeholder in and out counts match on all 50, and every restored span is byte identical to its original
Stop and report if	IndicTrans2 alters or drops placeholders. Report the pattern before working around it

12. Cross check the translation against the published MAPS instances

MAPS released translated SWE-bench Verified instances on Hugging Face at Fujitsu-FRE/MAPS, including Hindi. Find which of our 50 instances appear in their set.

For the overlap, compare their translated instruction against our translated_restored. You are not looking for identical text, you are looking for whether protected spans survive in both.

Write results/maps_overlap.csv with columns instance_id, in_maps, our_spans_intact, maps_spans_intact.

Produce	results/maps_overlap.csv
Done when	The overlap count is known and span survival is compared on it
Stop and report if	The overlap is empty. That is worth knowing and worth reporting, since it changes what the comparison can do

13. Synthesize the audio

Generate audio for each translated instruction using an Indian language synthesis system, with three different speaker identities per instance. Write audio/{instance_id}_{speaker_id}.wav, mono, at the sample rate the recognition system expects, and record that rate in config.md.

The synthesis system and the recognition system must come from independent model families. Write one paragraph in config.md naming both and justifying the independence. If they share a family, the round trip will look artificially clean and the result is worthless.

Produce	audio/ with 150 clips, plus the independence paragraph in config.md
Done when	All clips generated and the independence justification written
Stop and report if	You cannot find two independent families, or synthesis fails on instructions containing English spans

14. Check the audio actually says what it should

Run the naturalness scoring and word error rate over all clips. Then listen to a sample of 20 clips yourselves and mark, for each, whether the instruction is intelligible and whether every protected span is audible and correct.

Write results/fidelity.csv with columns clip, naturalness, wer, human_intelligible, spans_audible.

This is the step people skip. If the audio does not say what we think it says, every number after it is void.

Produce	results/fidelity.csv and the sampled human check
Done when	All clips scored and 20 checked by ear by both of you
Stop and report if	Protected spans are not audible in the sample. Synthesis is then producing the result, not the agent

15. Run recognition and compute preservation

Run the recognition system over every clip. Write transcripts/{instance_id}__{speaker_id}.json with keys instance_id, speaker_id, transcript.

Now compute identifier preservation rate at each stage, exactly as defined in spans/metric.md. The stages are: after spokenization, after translation and restoration, and after recognition.

Write results/preservation.csv with columns stage, span_type, spans_total, spans_surviving, preservation_rate.

What you are looking for is where the line drops. A steady decline across all stages and a cliff at one stage mean different things, and the cliff is the finding.

Produce	transcripts/ and results/preservation.csv
Done when	Preservation is computed at all three stages, broken down by span type
Stop and report if	Preservation is flat and high across every stage. That is a null result, it is reportable, and Ch decides what follows

16. Run C2, the translated text condition

Run SWE-agent on translated_restored as text, no audio involved, three runs. Write results/c2.csv.

Do not assume this comes back near zero loss. The brief explains why: in the MAPS per-language table, Hindi sits below the English baseline on the software portion, and it is the weakest language there.

Produce	results/c2.csv
Done when	Three runs complete and the mean is compared against C1
Stop and report if	C2 is far below C1, meaning the loss is in translation rather than speech. Report immediately, it changes the paper

17. Run C3, the cascade

Take the recognition transcript and pass it to SWE-agent unchanged, as its issue text. The agent is not modified in any way. Three runs. Write results/c3.csv.

C2 minus C3 is the speech channel loss, which is the central number of the probe.

Produce	results/c3.csv and the C2 minus C3 figure in notes.md
Done when	Three runs complete and the difference is computed
Stop and report if	The agent errors out on transcript input rather than failing to resolve. That is a plumbing problem, not a result

18. Run C4, the omni arm, and record language support

Pass the audio directly to an omni model with tool calling enabled, no intermediate transcript. Three runs. Write results/c4.csv.

Separately, for every omni model you can access, record whether it accepts the probe language at all. Write results/omni_support.csv with columns model, version, language, accepted, evidence, where evidence is something you observed when you tried it, not a claim from the vendor's documentation.

A model that does not support the language is recorded as unsupported. That is a result, not a gap in the table.

Produce	results/c4.csv and results/omni_support.csv
Done when	Either C4 runs, or the reason it cannot run is recorded with evidence
Stop and report if	No accessible omni model accepts the language. Record it and report, do not substitute a different language to make the arm run

Part D. What to send Ch

One message, with two tables in it, and nothing else. Not a folder of files, not a link to a repository. The tables go in the message body.

Table A, resolve rate by condition. One row per condition, C1, C1b, C2, C3, C4. Columns: condition, run 1, run 2, run 3, mean, standard deviation, instances resolved out of 50.

Table B, identifier preservation by stage. One row per stage. Columns: stage, span type, spans total, spans surviving, preservation rate.

Under the tables, three short paragraphs. What the drop from C1 to C3 was, and how it compares against the two published reference points, the MAPS figure for the same language and the per model English text to voice gap in [3]. Where the preservation line dropped, if it dropped. What broke, what you worked around, and what you could not close.

Then stop. No further building until you hear back. This is the point the brief calls the MUST, and it gates both workstreams.

Things you do not decide

These are in the brief under Decisions to make. They come back as proposals with the alternatives you rejected and what each one costs, not as choices you have already acted on.

Which language to run after the probe, and whether to go deep in one or across several. Whether the script variant arm returns. Whether the arm with instructions spoken by real developers returns. How the title is written. Whether this is positioned under the BhashaSE line or as its own thread. What the paper claims if the probe comes back flat.

If you reach one of these and cannot form a proposal, say so, and say what you would need in order to form one. That is a complete and acceptable answer.

AI Disclosure

The original ideas, analysis, and research synthesis in this report are the intellectual contribution of the authors. AI-assisted tools were used to enhance the clarity and presentation of the writing.